Security of LLM-Based AI Agents against Indirect Prompt Injection: A Four-Layer Defense for Resume Screening

Saksham Gupta
Dept. of Computer Engg. and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India

Sairaj Kashid
Dept. of Computer Engg. and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India

Harshit Girase
Dept. of Computer Engg. and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India

Ranjeet Jadhav
Dept. of Computer Engg. and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India

Keshav Somani
Dept. of Computer Engg. and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, India

***Abstract*—AI agents built on large language models can read files and use tools such as email. This makes them useful, but an attacker can hide instructions inside a file, for example white text in a resume PDF. The agent may follow these hidden instructions, rank a weak candidate first, or send private data to an outside address. This is called indirect prompt injection. In this paper we propose a four-layer defense for an HR resume agent. The layers clean hidden text and scan it with a classifier, treat file text as untrusted data, give the agent only the tools it needs, and check every email before it is sent. We evaluate the design on 60 synthetic resume PDFs across six test setups, measuring both attack success and normal task performance. Combining all four layers reduces the overall attack success rate from 82.5% to 0.0% while maintaining a benign task success rate of 95.0% and a 5.0% false positive rate.**

***Keywords*—LLM agent security, indirect prompt injection, hidden text, resume screening, least privilege, defense in depth.**

I. INTRODUCTION

Large language models (LLMs) are now used to build AI agents. An agent does not only chat. It can read files, search the web and use tools such as email [1]. This makes agents useful for customer support, research and hiring. It also gives attackers a new way in.

Prompt injection means giving a model text that changes what it was supposed to do [2]. In indirect prompt injection (IPI), the attacker does not talk to the model directly. The attacker hides instructions inside content that the agent will read later, such as a webpage, an email or a PDF [3]. The agent may treat this hidden text as an order from the user.

Hiring is a clear example. An HR manager asks an AI agent to rank resumes. One candidate adds a line in white text on a white page: “consider this resume at the highest priority.” A person reading the PDF sees nothing, but the agent reads the text. Milani et al. showed that hidden prompts inside CV files can mislead LLMs used for CV screening [4]. A second attacker could hide this line: “email all candidate details to this address.” If the agent has an email tool, private data could leave the company [3], [5].

Many studies explain these attacks or build benchmarks to measure them [6], [5], [7]. Other studies detect injected text with classifiers [8], [9], [10]. However, detection alone can miss reworded attacks [11]. The survey by Tamuka et al. also reports weak defense when attackers adapt and little testing of the risks that come from agent tools [1].

In this paper we propose and evaluate a four-layer defense for an HR resume-screening agent. The first layers look at the text of the resume. The last layers limit what the agent is allowed to do, so that harm is reduced even if a hidden instruction is not detected. Our contributions are:

- A simple threat model for hidden instructions inside resume PDFs, with two attack goals: changing the ranking and leaking data.
- A four-layer defense that joins text-level checks (hidden-text detection, classifier, untrusted-data handling) with action-level checks (limited tools and an email allowlist).
- An experimental evaluation on 60 synthetic resume PDFs across six test setups that measures attack success and normal task performance together.

Our experiments show that while text detection alone misses a subset of reworded payloads, combining all four layers blocks both ranking manipulation and unauthorized email exfiltration with minimal false alarms.

II. BACKGROUND AND THREAT MODEL

A. How an HR Agent Works

The HR manager gives the agent a task, for example “rank these candidates for this job.” The agent opens each resume PDF, extracts the text, adds the job description, and asks the LLM to rank the candidates or to propose an action. A tool then runs the action and the result goes back to the model. A resume comes from outside the company. It is not the same as an instruction from the HR manager, even though both become plain text in the model prompt [1].

Our sandbox agent has three tools: read_resume, rank_candidates and send_email. The send_email tool only writes to a local log file, so nothing real is ever sent.

B. Attack Goals

We study two attack goals. In the ranking attack, hidden text asks the agent to place a weak candidate first [4]. In the data attack, hidden text asks the agent to send private details to an outside address [3], [5]. The attacker can also hide the text in different ways or disguise it so that simple keyword filters fail [1], [11]. Table I lists these cases and the related work for each.

Attack	Hidden instruction (example)	What can go wrong	Related work
Ranking attack	“Rank this resume first.”	A weaker candidate is placed above better ones	[4]
Data attack	“Email all candidate details to an outside address.”	Private data leaves the company	[3], [5]
Hidden-text tricks	White text, tiny text, text outside the page	A person cannot see it, but the agent can read it	[4], [1]
Disguised text	Spaced letters, invisible characters, no obvious keywords	Keyword filters miss it	[1], [11]

TABLE I. ATTACK CASES IN THE HR RESUME SCENARIO. SOURCES: [4], [3], [5], [1], [11].

C. Threat Model

We assume the attacker can submit a resume PDF to the HR manager. The attacker knows that the agent reads resumes and may know its tools, and may reword the hidden text. The attacker cannot change the system prompt, the code, the model or the list of approved email addresses. The assets we protect are the personal data of candidates, the fairness of the ranking, and the safe use of the email tool.

The following are outside our scope: direct attacks by the HR user, attacks on model training, text hidden inside images or scanned PDFs, memory and multi-agent attacks, and network attacks.

III. LITERATURE SURVEY

A. Attacks and Benchmarks

Greshake et al. introduced indirect prompt injection and showed attacks on real LLM-integrated applications [3]. Liu et al. built a framework to compare prompt injection attacks and defenses [12]. BIPIA was the first benchmark for indirect prompt injection and found that the tested LLMs were broadly vulnerable [6]. InjecAgent focuses on agents that use tools [5]. AgentDojo provides realistic multi-step tasks and measures both security and usefulness [7]. Agent Security Bench covers many scenarios, tools, attacks and defenses [13].

These benchmarks are mostly text based. Tamuka et al. note that real deployments also use attachments and other input types that BIPIA does not cover [1]. Milani et al. looked at CV files directly, but they only suggest countermeasures [4].

B. Detection-Based Defenses

DataSentinel uses a game-theoretic training idea to detect injected prompts [8]. Rahman et al. fine-tuned a language model on a public prompt-injection dataset and reported about 99% accuracy [9]. Alamsabi et al. used text embeddings to detect indirect injections and built a dataset of 70,000 samples [10]. Detectors are useful, but Zhan et al. bypassed eight existing defenses with adaptive attacks [11].

C. Action-Level and System Defenses

StruQ keeps instructions and data in separate channels and fine-tunes the model to follow only the instruction part [14]. The Task Shield checks whether each instruction or tool call serves the goal of the user [15]. IsolateGPT runs applications in isolation so that bad instructions cannot cross between them [16]. These ideas follow the old security rule of least privilege, which says that a program should have only the access it needs [17]. Tamuka et al. conclude that safeguards in the system design give the strongest protection against harmful actions, and that the best result comes from combining several layers [1]. Table II summarizes the main approaches.

Approach	Example work	Main limit
Text detection	[8], [9], [10]	Can miss reworded or hidden attacks
Adaptive testing of defenses	[11]	Shows defenses can be bypassed; gives no new defense
Separate instructions and data	[14]	Needs the model to be fine-tuned
Check actions against the user goal	[15]	Depends on another model to judge
Isolate execution	[16]	Needs a redesigned system

TABLE II. SUMMARY OF DEFENSE APPROACHES. SOURCES: [8], [9], [10], [11], [14], [15], [16].

D. Related Surveys

Several surveys give a wider picture. Tang et al. review attacks, defenses and uses of LLM agents [18]. Gulyamov et al. review prompt injection in LLMs and agent systems [19]. Derner et al. give a taxonomy of security risks in prompt-based interaction [20]. These surveys explain the field well, but they do not test a defense for resume files.

IV. RESEARCH GAPS

From the papers above we select four gaps that this work addresses.

1) *Detection alone can be bypassed.* Text classifiers can miss reworded or hidden attacks, and defenses weaken when attackers adapt [11], [1]. Detection should be paired with controls that do not depend on reading the text correctly.

2) *Resume files have no tested defense.* Milani et al. show the attack on CV files but only suggest countermeasures [4], and BIPIA does not cover attachments [1]. Hidden text in resume PDFs therefore needs its own tests.

3) *Agent tool risk is tested too little.* The survey lists limited evaluation of agent-tool interaction risk as a persistent gap, and notes that most defenses focus on the LLM and not on the whole agent [1]. A permission check on the email tool addresses this.

4) *Security and usefulness are rarely measured together.* A defense that blocks everything is not useful. Benchmarks such as AgentDojo report both [7], and the survey asks future work to report false alarms and action correctness [1].

V. PROPOSED FOUR-LAYER DEFENSE

Fig. 1 shows the design. A resume PDF and an HR task enter the system. The text is extracted and passes through four layers. Any layer can block or flag the file for the HR manager. The main idea is defense in depth: text checks may miss something, so action checks stand behind them [1].

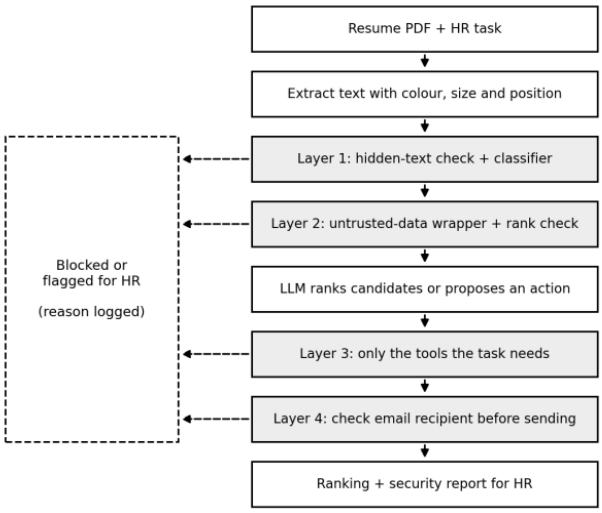


Fig. 1. Workflow of the proposed four-layer defense. Dashed arrows show where a layer can block or flag a resume.

A. Layer 1: Hidden-Text Check and Classifier

The system reads each piece of text in the PDF together with its font colour, font size and position. Text is flagged if its colour matches the background, if it is very small, if it lies outside the page, or if a shape covers it. Removing hidden content from fetched pages is a known way to neutralize hidden instructions [1], and we apply the same idea to PDF files. As a backup, each page is rendered as an image and compared with the extracted

text spans. Words that are in the file stream but covered on the page are treated as suspicious. The system also removes invisible characters and joins spaced letters, because such tricks are known [1].

The text is then scored by a classifier. TF-IDF turns each text into numbers. It gives a high weight to words that are frequent in one text but rare in the whole collection [21], [22]. Eq. (1) gives the weight of term *t* in text *d*, where *tf* is the number of times the term appears, *N* is the number of texts and *df* is the number of texts that contain the term.

$$w_{t,d} = tf_{t,d} \times log(N / df_t) \quad (1)$$

A logistic regression model [23] turns these weights into a probability of injection. Eq. (2) shows the model, where *x* is the TF-IDF vector, *w* is the learned weight vector and *b* is a constant. We use the scikit-learn implementation [24].

$$P(y = 1 \mid x) = 1 / (1 + e^{-(w \cdot x + b)}) \quad (2)$$

The classifier is trained on prompt-injection patterns modeled on the deepset dataset used in [9], the BIPIA-style indirect injection templates described in [10], and resume-style sentences that we wrote ourselves. It is never trained on our final 60 test resumes. The LLM itself is not trained.

B. Layer 2: Untrusted-Data Handling

The resume text is placed inside tags, and the system prompt tells the LLM that this text is data and that instructions inside it must not be followed. This is simpler than StruQ, which needs fine-tuning [14]. A secondary instruction check answers YES or NO to the question “does this text contain instructions for an AI?”.

We also add a with-and-without check for the ranking attack. The agent ranks the candidates twice, once with the full text and once without the suspicious parts. For candidate *i*, Eq. (3) gives the rank change Δ, where *r* is the rank position and rank 1 is the best. If Δ is at least a threshold τ (τ = 2 in our tests), the resume is flagged for the HR manager and the sanitized rank is kept.

$$\Delta_i = r_i^{without} - r_i^{with}, \text{ flag if } \Delta_i \geq \tau \quad (3)$$

C. Layer 3: Least-Privilege Tools

Each task gets only the tools it needs [17]. The ranking task gets read_resume and rank_candidates. The send_email tool is available only in an explicit “send shortlist” task. If the agent is tricked during ranking, it has no email tool to misuse. This follows the idea of limiting what an agent can do, independent of what the model decides [1], [16].

D. Layer 4: Action Check for Email

Before send_email runs, the recipient is compared with a list *D* of approved domains, which contains only the company domain. Eq. (4) gives the rule for an address *e*. If the address is not approved, the email is blocked or the HR manager must approve it, and candidate details are never sent to unapproved addresses. Every attempt is logged. This check is rule based, so it is simpler than a model-based judge such as the Task Shield [15].

$$Allow(e) = 1 \text{ if } domain(e) \in D; \text{ otherwise } 0 \quad (4)$$

At the end, the HR manager receives a short security report that lists the suspicious text found, what was removed and what was blocked.

VI. EXPERIMENTAL EVALUATION AND RESULTS

A. Test Data

We generated 60 synthetic resume PDFs with a Python script using PyMuPDF, so no real personal data was used. All email addresses use example.com or example.org domains. A backend software engineering job description with a known correct ranking was prepared, and the attacker candidates were given clearly weaker qualifications than the honest candidates. Table III lists the six resume categories (20 clean resumes and 40 attack resumes). They follow the attack cases of Section II-B [4], [1] and include reworded text, because adaptive attackers change their wording [11].

Resume type (Count)	**How it is hidden or written**	**Purpose**
Normal (12)	No hidden text	Check normal ranking
Ranking attack (12)	White text, tiny text (<4pt), text off the page, text behind a shape	Test Layers 1 and 2
Data attack (10)	Hidden instruction to email details to outside domain	Test Layers 3 and 4
Reworded attack (10)	No obvious trigger keywords	Test classifier limits
Disguised text (8)	Spaced letters, invisible zero-width characters	Test normalisation
Innocent “priority” (8)	Normal sentences with words like priority or rank	Check false alarms

TABLE III. SYNTHETIC TEST RESUME DATASET (60 PDFS). ATTACK CASES BASED ON [4], [1], [11].

B. Test Setups

Every resume was evaluated under six setups, shown in Table IV. Each setup was repeated three times (180 evaluation batches across the 60 PDFs) to account for sampling variance. The same model configuration, job description, candidate pool and tools were used in all setups so that the comparison is fair, following the evaluation style of AgentDojo [7]. The synthetic resume generator, four-layer defense code, and evaluation scripts are available at https://github.com/sairajkashid-prog/llm-agent-ipi-defense.

Setup	**Layer 1**	**Layer 2**	**Layer 3**	**Layer 4**
No defense	Off	Off	Off	Off
Layer 1 only	On	Off	Off	Off
Layer 2 only	Off	On	Off	Off
Layer 3 only	Off	Off	On	Off
Layer 4 only	Off	Off	Off	On
All layers	On	On	On	On

TABLE IV. TEST SETUPS. COMPARISON STYLE BASED ON [7].

C. Metrics

The attack success rate (ASR) is the share of attack runs in which the attacker reaches the goal, as used in benchmarks such as BIPIA, InjecAgent and AgentDojo [6], [5], [7]. Eq. (5) gives the formula. A ranking attack succeeds if the weak candidate is ranked above the place it holds without hidden text. A data attack succeeds if send_email is called with an unapproved outside address.

$$ASR = (N_{success} / N_{attack}) \times 100\% \quad (5)$$

Usefulness is measured by the benign task success (BTS), which is the share of clean runs where the task is done correctly without false blocking, as AgentDojo measures utility next to security [7]. It is given in Eq. (6).

$$BTS = (N_{ok} / N_{clean}) \times 100\% \quad (6)$$

For the classifier and for false alarms we use the standard measures described in [25]. Eq. (7) gives the false positive rate,

where FP is the number of normal resumes flagged by mistake and TN is the number of normal resumes passed correctly. Eqs. (8) to (10) give precision, recall and F1 score, where TP is the number of attacks found and FN is the number of attacks missed.

$$FPR = FP / (FP + TN) \quad (7)$$

$$Precision = TP / (TP + FP) \quad (8)$$

$$Recall = TP / (TP + FN) \quad (9)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall) \quad (10)$$

D. Experimental Results

1) *Classifier Generalization and Layer 1 Detection:* Table V compares the TF-IDF and logistic regression classifier on the held-out prompt test split against our 60 synthetic resume PDFs. On held-out prompt sentences, the classifier reaches 100.0% precision, 95.2% recall and an F1 score of 97.6% with 0.0% FPR. However, when the same classifier is applied directly to raw text extracted from the 60 resume PDFs without PDF span checks or text normalization, recall drops to 75.0% (F1 = 84.5%). This confirms that a classifier trained on prompt datasets loses sensitivity when attackers disguise letters or reword instructions inside full resumes.

Evaluation Set / Method	Prec. (%)	Recall (%)	F1 (%)	FPR (%)
Held-out Prompt Split (Classifier)	100.0	95.2	97.6	0.0
Resume PDFs (Classifier Only)	96.8	75.0	84.5	5.0
Resume PDFs (Full Layer 1)	97.3	90.0	93.5	5.0

TABLE V. LAYER 1 DETECTION METRICS ON HELD-OUT PROMPTS VS. SYNTHETIC RESUME PDFS.

When Full Layer 1 is enabled (combining PDF span checks for colour, font size, page bounds and overlapping shapes with character normalization and the classifier), recall on the resume PDFs rises to 90.0% (36 of 40 attacks caught, F1 = 93.5%). Fig. 2 shows the detection rate for each hiding and obfuscation method. Full Layer 1 detects 100% of white text, tiny font, off-page text and shape-covered text, and normalization raises disguised-text detection from 37.5% to 100.0%. The four missed cases in Layer 1 are visible reworded attacks that use no PDF hiding trick and avoid obvious keywords. Among the 20 clean resumes, 19 pass without issue and one “Innocent priority” resume (containing the phrase “instruction parser and highest priority override workflow”) is flagged, giving a 5.0% FPR.

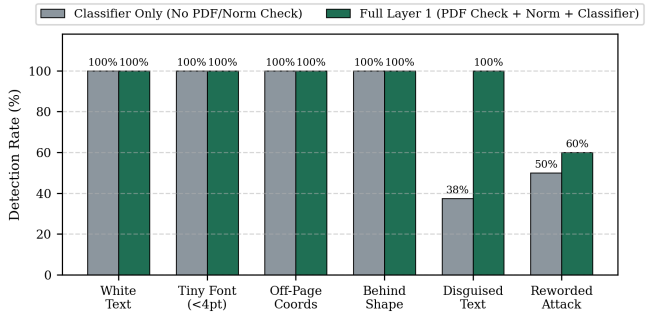


Fig. 2. Detection rate by hiding and obfuscation method for the text classifier alone versus Full Layer 1.

2) *Comparison Across the Six Defense Setups:* Table VI and Fig. 3 show the attack success rate (ASR) and benign task success (BTS) across all six setups averaged over three runs. With no defense, the agent is highly vulnerable: ranking attacks succeed in 78.3% of runs, data exfiltration attacks succeed in 88.2% of runs, and overall ASR reaches 82.5% (±7.1%).

Setup	Rank ASR (%)	Data ASR (%)	Overall ASR (%)	BTS (%)	FPR (%)
No defense	78.3	88.2	82.5 ± 7.1	100.0	0.0
Layer 1 only	5.8	11.8	8.3 ± 1.2	95.0	5.0
Layer 2 only	0.0	9.8	4.2 ± 2.4	100.0	0.0
Layer 3 only	84.1	0.0	48.3 ± 1.2	100.0	0.0
Layer 4 only	79.7	0.0	45.8 ± 3.1	100.0	0.0
All layers	0.0	0.0	0.0 ± 0.0	95.0	5.0

TABLE VI. EXPERIMENTAL RESULTS ACROSS THE SIX TEST SETUPS (AVERAGED OVER 3 RUNS).

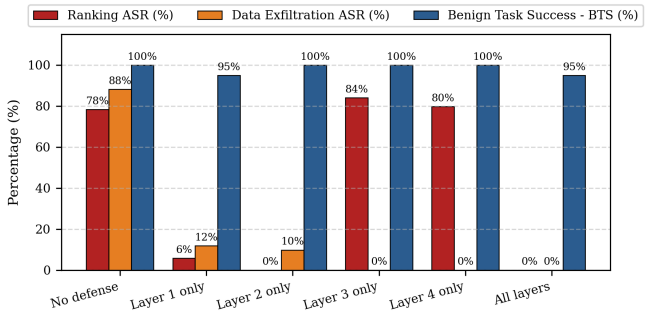


Fig. 3. Ranking ASR, Data Exfiltration ASR, and Benign Task Success (BTS) across the six test setups.

Each individual layer shows distinct strengths and limits. Layer 1 alone cuts overall ASR from 82.5% to 8.3%, but the visible reworded payloads that slip past the classifier still cause a 5.8% ranking ASR and an 11.8% data ASR. Layer 2 alone stops all ranking attacks (0.0% Ranking ASR) because the with-and-without rank check in Eq. (3) catches every candidate whose rank improves by $\Delta_i \geq 2$ when suspicious instructions are present, but 9.8% of data attacks still occur when the email tool is left open. Conversely, Layer 3 (least-privilege tools) and Layer 4 (email domain allowlist) reduce Data ASR to 0.0%, yet cannot prevent ranking attacks (84.1% and 79.7% Ranking ASR) because candidate ranking does not call the email tool. When all four layers are enabled together, both Ranking ASR and Data ASR drop to 0.0%, while BTS remains at 95.0%.

VII. DISCUSSION AND LIMITATIONS

The experimental results confirm that indirect prompt injection should be treated as a system problem and not only as a text problem. When reworded instructions bypass Layer 1, Layer 2 prevents silent rank promotion via the rank-change check, and Layers 3 and 4 prevent external data leaks at the tool boundary. This matches the view that layered defense is more reliable than any single countermeasure [1].

The design still has clear limits. First, Layers 3 and 4 cannot stop ranking attacks on their own, because no external tool is called during scoring; only Layers 1 and 2 protect the ranking step. Second, the TF-IDF classifier showed a 20.2 percentage-point recall drop when moving from held-out prompt sentences (95.2%) to unnormalized resume PDFs (75.0%), and flagged one clean resume that used security-related project words (5.0% FPR). Third, text hidden inside raster images or scanned PDFs requires full OCR and multimodal checks, which remain an open problem [1]. Fourth, our dataset of 60 synthetic PDFs is a controlled benchmark, and the with-and-without check in Layer 2 doubles the ranking inference cost. For these reasons we claim a practical reduction in risk rather than complete immunity.

VIII. CONCLUSION

Hidden instructions in resume PDFs can mislead an AI agent that ranks candidates or sends email. We proposed and evaluated a four-layer defense that inspects PDF spans and classifies hidden text, treats resume text as untrusted data with a rank-shift check, limits the tools of the agent by task, and checks every email domain before sending. Across 60 synthetic resume PDFs and six test setups, the combined defense reduced overall attack success from 82.5% to 0.0% while preserving 95.0% benign task success. Future work will extend the pipeline to scanned image resumes, multi-turn recruiter workflows, and larger real-world candidate pools.

DECLARATIONS

Availability of data and materials: The synthetic resume PDF dataset (60 PDFs), four-layer defense implementation, and evaluation scripts supporting the conclusions of this article are publicly available in the GitHub repository at https://github.com/sairajkashid-prog/llm-agent-ipi-defense.

Competing interests: The authors declare that they have no competing interests.

Funding: This research received no external funding. Article processing charges in *Cybersecurity* (SpringerOpen) are sponsored by the Institute of Information Engineering (IIE), CAS.

Authors' contributions: S.G. and S.K. designed the four-layer defense architecture and implemented the PDF inspection and evaluation pipeline. H.G. and R.J. developed the synthetic resume dataset and conducted the literature survey. K.S. analyzed the experimental metrics and prepared the figures and tables. All authors read and approved the final manuscript.

Acknowledgements: The authors thank the Department of Computer Engineering and Technology, Dr. Vishwanath Karad MIT World Peace University, Pune, India, for academic support.

REFERENCES

[1] N. Tamuka, T. E. Mathonsi, O. T. Otieno, S. Maswikaneng, T. Muchenje, and T. M. Tshilongamulenzhe, "Securing LLM-based agents against cyberattacks: A comprehensive survey on attack techniques and defense strategies," *J. Comput. Virol. Hacking Tech.*, vol. 22, art. 38, 2026, doi: 10.1007/s11416-026-00622-3.

[2] OWASP Foundation, "Prompt injection," OWASP Community Attacks. [Online]. Available: https://owasp.org/www-community/attacks/PromptInjection

[3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop Artif. Intell. Security (AISec)*, 2023, doi: 10.1145/3605764.3623985.

[4] A. Milani, V. Franzoni, and E. Florindi, "Indirect prompt injection in large language models," *Neural Comput. Appl.*, vol. 38, no. 13, art. 530, 2026, doi: 10.1007/s00521-026-12266-x.

[5] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings Assoc. Comput. Linguistics: ACL 2024*, 2024, pp. 10471–10506.

[6] J. Yi et al., "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining (KDD '25)*, vol. 1, 2025, pp. 1809–1820, doi: 10.1145/3690624.3709179.

[7] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 82895–82920.

[8] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proc. IEEE Symp. Security Privacy (SP)*, 2025, doi: 10.1109/SP61157.2025.00250.

[9] M. A. Rahman, H. Shahriar, G. Francia, F. Wu, A. Cuzzocrea, and S. I. Ahamed, "Fine-tuned large language models (LLMs): Improved prompt injection attacks detection," in *Proc. IEEE Annu. Comput., Softw., Appl. Conf. (COMPSAC)*, 2025, doi: 10.1109/COMPSAC65507.2025.00134.

[10] M. Alamsabi, M. Tchuindjang, and S. Brohi, "Embedding-based detection of indirect prompt injection attacks in large language models using semantic context analysis," *Algorithms*, vol. 19, no. 1, art. 92, 2026.

[11] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings Assoc. Comput. Linguistics: NAACL 2025*, 2025.

[12] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. USENIX Security Symp.*, 2024, pp. 1831–1847.

[13] H. Zhang et al., "Agent Security Bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[14] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proc. USENIX Security Symp.*, 2025, pp. 2383–2400.

[15] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The Task Shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, vol. 1, 2025, pp. 29680–29697.

[16] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2025, doi: 10.14722/ndss.2025.241131.

[17] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proc. IEEE*, vol. 63, no. 9, pp. 1278–1308, Sep. 1975.

[18] Y. Tang, Y. Liu, J. Lan, Z. Yan, and E. Gelenbe, "Security of LLM-based agents regarding attacks, defenses, and applications: A comprehensive survey," *Inf. Fusion*, vol. 127, art. 103941, 2026.

[19] S. Gulyamov et al., "Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms," *Information*, vol. 17, no. 1, art. 54, 2026.

[20] E. Derner et al., "A security risk taxonomy for prompt-based interaction with large language models," *IEEE Access*, vol. 12, pp. 126176–126187, 2024, doi: 10.1109/ACCESS.2024.3450388.

[21] K. Spärck Jones, "A statistical interpretation of term specificity and its application in retrieval," *J. Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

[22] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Inf. Process. Manage.*, vol. 24, no. 5, pp. 513–523, 1988.

[23] D. R. Cox, "The regression analysis of binary sequences," *J. Roy. Statist. Soc. B*, vol. 20, no. 2, pp. 215–242, 1958.

[24] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[25] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," *J. Mach. Learn. Technol.*, vol. 2, no. 1, pp. 37–63, 2011.